

2026-08-31 更新:λ 用训练集重新调优 + ChainRAG/GRITHopper 复现对比

本周两件事:1) 不在验证集上直接调 λ,改在训练集子集上重新扫描,避免"调参和报告用同一份数据" 的隐患;2) 复现 ChainRAG、GRITHopper 这两个跟本项目思路接近的工作,对比检索召回和最终答案效果。

一、λ 用训练集重新调优

1.1 动机

现有 gate v3 D 组的 λ=0.50 是直接在验证集(dev)上扫出来的——虽然只是一维标量搜索、风险有限, 但更干净的做法是拿一份模型完全没在上面调过参的数据去选超参,再回到 dev 上验证,而不是让"调参" 和"最终报告的评测集"完全重合。这次改成在 **训练集子集** 上扫 λ,dev 只用来做最后的确认。

1.2 方法

- 从 MuSiQue train(GT decompose)按 K 分层抽样 3000 条(K=2/3/4 各 1000),生成对应的 **BART decompose**(不是 GT——BART 分解模型本身是在 train 上训练出来的,如果直接用 GT 或者在 BART-on-train 上测,会失真,这里特意确认过 BART 在这 3000 条 train 子集上的输出仍然带有真实 的生成噪声,不是被"记忆"到接近 GT 质量的干净文本,细节见对话记录)。
- 用当前完整方法(rawprefix + comparison_hint reader prompt)在这 3000 条上扫 λ ∈ [0.10, 0.80],步长 0.05。

1.3 结果:train 子集上的 λ 曲线

λ	recall@1	chain@1	EM	F1
0.10	0.7252	0.4710	0.4193	0.5005
0.20	0.7515	0.5133	0.4413	0.5271
0.30	0.7555	0.5183	0.4527	0.5414
0.40	0.7561	0.5217	0.4543	0.5426
0.50	0.7558	0.5223	0.4540	0.5425
0.60	0.7556	0.5250	0.4593	0.5460
0.65	0.7540	0.5220	0.4587	0.5461
0.70	0.7523	0.5210	0.4580	0.5453
0.80	0.7500	0.5163	0.4567	0.5437

EM/F1 在 0.60-0.65 见顶,0.70 之后四项指标一起往下掉,是一个真实的内部最优区间,比 dev 上原来 调出的 0.50 更高。

1.4 dev 全量验证:0.50 → 0.60

用 λ=0.60 在三个数据集的 dev 全量上直接跑了一遍验证:

数据集	指标	λ=0.50	λ=0.60	差值
MuSiQue	recall@1/chain@1/EM/F1	0.7018/0.5143/0.4427/0.5386	0.6994/0.5118/0.4386/0.5341	全部小幅下降(EM -0.41pp)
2WikiMultihopQA	同上	0.9289/0.8613/0.5982/0.6770	0.9316/0.8685/0.6023/0.6819	全部小幅上升(EM +0.41pp)
HotpotQA	同上	0.6595/0.5567/0.5517/0.6870	0.6620/0.5610/0.5519/0.6883	全部小幅上升

结果好坏参半,但方向很关键:λ=0.50 当初主要是针对 **MuSiQue** 调出来的,而 **0.60** 恰好在 **MuSiQue** 上小幅下降、却在另外两个完全没参与调参的数据集上都有提升——说明 0.60 是一个跨数据集更稳健的 取值,不是对单一数据集过拟合出来的数字。采纳 λ=0.60 为新的

默认值 (run_retrieval_exp_wavefront_gate_v3_rawprefix.py 的 --lambda-gate 默认值已同步更新)。

1.5 最终对比:不用 gate(baseline) vs 新 $\lambda=0.60$ (完整方法)

数据集	方案	recall@1	recall@3	chain@1	EM	F1
MuSiQue	baseline(无 gate)	0.6003	0.7611	0.3335	0.3757	0.4652
	+ gate ($\lambda=0.60$, 完整方法)	0.6994	0.8435	0.5118	0.4386	0.5341
2WikiMultihopQA	baseline (无 gate)	0.8123	0.9021	0.6047	0.4544	0.5230
	+ gate ($\lambda=0.60$, 完整方法)	0.9316	0.9842	0.8685	0.6023	0.6819
HotpotQA	baseline (无 gate)	0.5746	0.7760	0.3361	0.4918	0.6163
	+ gate ($\lambda=0.60$, 完整方法)	0.6620	0.8701	0.5610	0.5519	0.6883

三个数据集加 gate 之后全指标提升明显,MuSiQue 上 EM +6.3pp/F1 +6.9pp,2Wiki 上 EM +14.8pp/F1 +15.9pp,HotpotQA 上 EM +6.0pp/F1 +7.2pp——gate 融合信号(+ rawprefix + reader prompt 修复 + 本次重新调优的 λ)相对纯余弦检索基线的收益依然稳固。

二、ChainRAG / GRITHopper 复现对比

克隆并本地化了两个跟本项目思路接近的开源工作——ChainRAG(ACL 2025)、GRITHopper(EACL 2026) ——用本项目自己的三个数据集(MuSiQue/2WikiMultihopQA/HotpotQA,distractor setting,不是它们 各自原本的评测设定)重新跑一遍,统一口径,跟 D 组做三方对比。目录:chainrag/、grithopper/ (都在工作区根目录,独立 git,不影响本仓库)。三个数据集、三个方法全部跑完。

2.1 ChainRAG 的算法流程,配一个真实例子

ChainRAG("Mitigating Lost-in-Retrieval Problems in Retrieval Augmented Multi-Hop QA")的核心 思路是:先把整篇 context 建成一张"句子图",每个子问题检索时不满足于第一次排序结果,不够就在图 上做实体连通扩展,直到"够用"或者预算耗尽。具体分四步:

1. 建句子图(每道题一次):把这道题的候选段落全部按句子切开,抽取每句话里的实体,句子之间 凡是共享实体就连一条边。这张图后面每个子问题共用。
2. 判断要不要拆、怎么拆(LLM 调用):先问一次"这题需要几步推理吗",需要的话再拆成子问题。
3. 逐个子问题处理(核心循环):
 - 种子检索:子问题过 embedding 模型跟这道题所有句子算 cosine,取 top-100(咱们 distractor setting 下这题往往总共就 60-100 句,这一步基本没筛掉什么,后面等于对全量候选 精排),cross-encoder 精排取 top-7。
 - 问 LLM 够不够答:够就直接生成这个子问题的答案,进入下一个子问题;不够就往下一步。
 - 图扩展:把种子句子在句子图里的 1-hop 邻居全部拉进 context,再问一次;还不够就扩到 2-hop、3-hop,直到够了或者攒够 3000 词预算,强行生成一个答案。
4. 汇总生成最终答案:所有子问题都处理完后,把"子问题+各自答案"的列表连同原始问题一起喂给 LLM,生成最终答案。

真实例子(MuSiQue dev,题目"Who led the military expedition in the province that borders Zhejiang to the south?",gold 答案 Chen Zheng,gold 段落是《Zhejiang》和《Hokkien》两篇):

ChainRAG 把这题拆成了:

```
子问题 1: "Which province borders Zhejiang to the south?"
子问题 2: "Who led the military expedition in that province?"
```

子问题 2 检索排序 ranked_titles 第一名正是《Hokkien》(gold 段落,检索本身是对的)。但《Hokkien》这篇原文里其实写了两次"military expedition"(677 年 Chen Zheng 那次平定叛乱,和 885 年 Wang Chao/Wang Shenzhi 那次平定黄巢之乱的余波),而这道题图扩展之后累积进 context 的句子平均有 10 个不同 title 的内容(约等于这题候选池的一半),两件事的描述都被塞进了最终生成 的输入里——ChainRAG 最终给出的答案是"Wang Chao and Wang Shenzhi",选错了年代更晚的那次,判为错误(EM=0)。检索精确命中了 gold,最终答案还是错的,根子出在图扩展"广撒网"策略往 context 里塞了太多互相竞争的候选事实,稀释了正确信号。

2.2 GRITHopper 的算法流程,配同一个例子

GRITHopper("Decomposition-Free Multi-Hop Dense Retrieval")的思路完全相反:不解构子问题, 用一个专门微调过的 7B 模型 (GritLM-7B 底座 + GritHopper 微调权重,训练数据含 MuSiQue、2Wiki、 HotpotQA、EX-FEVER、HoVer),直接拿原始问题 + 已检索到的证据去迭代检索:

1. **编码 query**:把"原始问题 + 目前为止检索到的证据"拼成一个 prompt,过模型编码成一个向量 (不区分这是第几跳,每一跳看到的都是"原始问题+累积证据",没有针对"这一跳该找什么"单独生成 过子问题)。
2. **算相似度、取 top-1**:跟这道题所有候选段落的向量算 cosine,取分数最高的一个,提交为这一跳 的检索结果。
3. **累积证据,重复**:把刚提交的段落加进"已检索证据"列表,回到第 1 步,直到跳数够了(咱们固定 用 gold 跳数 K,不用它自己的自动停止机制,避免"停不停"跟"检索准不准"两件事混在一起评测)。

同一道题上,GRITHopper 两跳都命中了 gold(《Zhejiang》→《Hokkien》),证据链跟 D 组一样精确, **最终答案正确地给出了"Chen Zheng"**(EM=1)——不过这道题只是众多例子里的一个,GRITHopper 整体 的答案准确率其实明显偏低,后面 2.5 节详细说为什么。

同样值得记录的一个反例(揭示"无需拆解"设计的一个结构性弱点):题目"Where in Zhejiang is the city where Protestants are especially notable?",gold 答案 Yongjia County,gold 段落是 《Sanjiang Church》和《Zhejiang》两篇。GRITHopper **两跳都检索到了同一篇《Zhejiang》**(没有 真正拿到第二篇《Sanjiang Church》)——因为每一跳的 query 都只是"原始问题+已有证据",没有一个明确指向"这一跳该找什么"的子问题,模型没能意识到还需要找一篇不同的、更具体的文章,在同一篇上 "打转"了。最终答案给出"Wenzhou"(从唯一检索到的 Zhejiang 那段里能抠出的最接近信息),但真正的 gold 答案"Yongjia County"在它压根没检索到的《Sanjiang Church》里,判为错误。这类"同一篇反复 命中、没有真正探索到新证据"的情况,是 GRITHopper recall@1 很高但"全覆盖率"明显更低的部分原因(见 2.4 节数字)。

2.3 统一 metric 的严格定义

跟三个方法都对齐用的是同一套定义,不是各自方法论文里的原始定义:

- **recall@1 / recall@3(顺序无关)**:对每一个检索决策单元(咱们是"跳"、ChainRAG 是"子问题"、 GRITHopper 是"迭代检索的每一步"),取它自己排序输出的候选列表,前 1 名 / 前 3 名里只要命中 **这道题需要的任意一个 gold 段落**(不要求对应哪个特定位置),就算命中;命中的决策单元数 / 决策单元总数 = recall@k。选"顺序无关"是因为它是唯一能同时套用在三方身上的:咱们的"跳"跟 gold 有数据集标注保证的位置对应关系,但 ChainRAG/GRITHopper 都没有这层保证(自己拆解/自己迭代, 跳数、顺序都不保证跟数据集标注对齐)。D 组这边专门改了 `run_retrieval_exp_wavefront_gate_v3_rawprefix.py`,新增 `full_pool_gold_rank_orderinvariant_*` 字段,用完整候选池(剪枝前)实时算出这个口径,不是事后拼凑的。
- **每题 gold 覆盖率 / 全覆盖率**:对每一道题,把它**整个 trace 里最终实际提交/使用过的证据**(D 组和 GRITHopper 是"每跳最终选定的那 1 个",K 跳就是 K 个;ChainRAG 是"每个子问题最终塞进 answer 生成 context 的所有句子来源 title",经过图扩展后往往是几十个)取并集,跟这道题的 gold 段落集合取交集,覆盖了几个 gold 就是几分之 gold 数;覆盖率=1.0(gold 一个不漏)记一次全覆盖。**这个指标 D 组和 GRITHopper 互相比(同样是"每跳 1 个,绝不多拿")**,但都不能直接跟 ChainRAG 的覆盖率相减比较——ChainRAG 的候选基数天然大出几倍(图扩展把大量非 top-1 的句子也算进去), 覆盖率高很大程度上是"候选多"带来的,不是"选得准"。
- **EM / F1**:标准的 HotpotQA 官方口径(`normalize_answer` 去冠词/标点/大小写 + token 重叠 F1),三个方法通用,可以直接比。三方**最终生成答案用的都是同一个本地 Llama-3.1-8B-Instruct** (D 组本来就用它;ChainRAG 的 `LLM.py`、GRITHopper 的 `run_answer_gen.py` 都改成调同一个本地 vLLM server),保证比的是"检索+推理设计"的差异,不是换了不同能力的生成模型。

2.4 完整三方对比(三个数据集全量)

数据集	方法	recall@1	recall@3	gold 覆盖率	全覆盖率	EM	F1
MuSiQue	D 组($\lambda=0.60$)	0.8552	0.9622	0.7733	0.5283	0.4377	0.5324
	ChainRAG	0.6716	0.8628	0.9587	0.9048	0.2764	0.3847
	GRITHopper	0.9058	0.9863	0.8207	0.6090	0.2793	0.3734
2WikiMultihopQA	D 组	0.9723	0.9970	0.9407	0.8799	0.6023	0.6819
	ChainRAG	0.9199	0.9785	0.9870	0.9696	0.4426	0.5220
	GRITHopper	0.9500	0.9964	0.8922	0.7751	0.5390	0.6267
HotpotQA	D 组	0.8988	0.9805	0.8532	0.7317	0.5519	0.6883
	ChainRAG	0.8032	0.9496	0.9864	0.9741	0.4598	0.6027
	GRITHopper	0.9658	0.9959	0.9464	0.9068	0.5546	0.6926

三个数据集结论一致:recall@1/@3 排名 GRITHopper > D 组 > ChainRAG;gold 覆盖率/全覆盖率 排名 ChainRAG > GRITHopper $\approx$ D 组(ChainRAG 靠图扩展"广撒网"占优,但这个指标口径跟前者不对等, 见 2.3);**EM/F1 排名 D 组明显领先,GRITHopper 和 ChainRAG 互有胜负、都明显落后于 D 组**—— 2Wiki/HotpotQA 上 GRITHopper 反超 ChainRAG(检索优势这次转化成了一部分答案优势),但 MuSiQue 上两者 EM 几乎打平(0.2793 vs 0.2764),说明检索优势转化成答案优势的程度并不稳定,答案合成 环节(有没有拆解脚手

架、context 干不干净)才是决定最终效果的关键变量。表格里两个模式最突出、 也最反直觉:**ChainRAG 覆盖率全场最高,EM/F1 却全场最低;GRITHopper 召回全场最高,EM/F1 依然 明显落后于 D 组**。这两个"高分指标换不来好答案"的现象,原因并不相同,下面分开看。

2.5 为什么 ChainRAG、GRITHopper 效果不如 D 组——两个方法各自的问题

2.5.1 ChainRAG 的问题:图扩展"广撒网"稀释了正确信号

ChainRAG 是三个方法里唯一完全不训练的方案(现成 embedding 模型 + 现成 cross-encoder,零参数 针对这三个数据集调整过),recall 本身也是三个里最低的——但它真正的问题不在召回,而在**即使 召回命中了 gold,答案还是可能错**。2.1 节的真实例子已经说明:子问题 2 的检索排序第一名正是 gold 段落《Hokkien》,但这篇原文里恰好写了两次"military expedition"(677 年 Chen Zheng 那次、 885 年 Wang Chao/Wang Shenzhi 那次),而图扩展为了保证"够用",把这道题平均约 10 个不同 title 的句子都拉进了最终生成的 context——两件事的描述被同时塞进输入,最终答案选错了年代更晚的那次 ("**Wang Chao and Wang Shenzhi**"),判为**错误**。检索精确命中,答案依然错,根子在图扩展这种"宁可 多拿、不做取舍"的策略,把正确信号淹没在了一堆同样看似相关的候选事实里——这也正好解释了它 gold 覆盖率全场最高(候选基数天然大)但 EM/F1 全场最低的矛盾组合。

2.5.2 GRITHopper 的问题:检索投入巨大,但没有拆解脚手架,答案合成负担全部后移

GRITHopper 的问题性质不同:它是三个方法里**检索最准的**,但答案效果同样明显落后于 D 组——MuSiQue 上 recall@1=0.9058(全场最高),EM=0.2793(全场最低,比 D 组的 0.4377 低 15.8 个百分点)。 检索最准、 答案最差,这个反差需要分两步解释:先说清楚它的检索为什么这么强,再说清楚为什么这份 检索优势没能带到答案上。

检索为什么强——三个方法的检索机制、 投入成本对比:recall 的排名(GRITHopper > D 组 > ChainRAG)基本跟投入程度成正比:

	ChainRAG	D 组(咱们)	GRITHopper
检索模型	现成的通用 embedding 模型 + 现成的通用 cross-encoder reranker,零训练(直接调用,没有针对这三个数据集训练过一点参数)	BAAI/bge-base-en-v1.5(1.1亿参数,通用 embedding 模型,零训练)+ 一个专门训练的小 gate(PCA+LR/PCA+双线性,参数量级是千到万,只在 MuSiQue 的隐藏状态上训练过)	GritLM-7B 底座(7B 参数)+ 整个模型专门微调过,训练数据覆盖 MuSiQue、2Wiki、HotpotQA、EX-FEVER、HoVer 五个数据集
是否见过这三个数据集	完全没见过(纯 prompt,零样本)	gate 在 MuSiQue 上训练过(GT decompose 的隐藏状态)	三个数据集全部在训练数据里
结构设计	bi-encoder 粗排 + cross-encoder 精排 + 实体图扩展(用图结构弥补检索不够准的问题,不追求一次选准)	bi-encoder 检索 + 小 gate 重排(一次选准,不做扩展/兜底)	专用模型直接编码"问题+累积证据"、一次选准(不做扩展/兜底,跟 D 组同一个设计哲学)
训练成本	无	低(小 gate,单卡几小时级)	高(7B 模型微调,需要多数据集训练流程,论文自己的训练成本远高于咱们这个 gate)

GRITHopper 是三个里唯一一个"针对多跳检索这个任务整体微调过 7B 级别模型"的方案,训练数据直接 覆盖了咱们评测用的这三个数据集——本质上是用重投入换来的检索精度优势,不是架构上有什么咱们和 ChainRAG 都没想到的巧思。D 组能排在中间,是因为底座检索器(BGE)虽然跟 ChainRAG 一样零训练,但 额外加了一个专门训练过的小 gate 做校正,用很小的训练投入换回了明显高于纯零训练方案 (**ChainRAG**) 的检索精度,这也是这个 gate 存在的意义所在。

检索优势为什么没能带到答案上,用一个真实例子说明咱们的 reader 输入长什么样(MuSiQue dev, 题目"Who led the military expedition in the province that borders Zhejiang to the south?"):

Reasoning trace:
Step 1:
Subquestion: What province borders Zhejiang to the south?
Selected evidence: "Zhejiang ... is bordered by ... Fujian province to the south ..."
Answer: Fujian

Step 2:
Subquestion: Who led the military expedition in Jiangxi

```
Selected evidence: "In 677 (during the reign of Emperor Gaozong), Chen Zheng (陳政), together with his son Chen Yuanguang (陳元光), led a military expedition to pacify the rebellion in Fujian. ... In 885, ... Wang Chao (王潮) and Wang Shenzhi (王審知), led a military expedition force to pacify the Huang Chao rebellion. ..."
```

Answer: NA

Re-read the original question before answering:

Who led the military expedition in the province that borders Zhejiang to the south?

Answer the original question directly using only the reasoning trace above.

Give a short span or phrase only (entity, date, number, or yes/no).

If the original question asks you to COMPARE two or more entities ..., the per-step answers above are the values used to make that comparison, not the final answer themselves ...

Output exactly one line:

Final answer: <short answer>

这个真实例子里有个很说明问题的细节:**hop 2 自己的短答案抽取失败了(Answer: NA)**——BART 把这一跳的子问题错误地写成了"in Jiangxi"(应该是 Fujian,BART 分解本身有误),导致短答案抽取 模块因为问题和证据对不上实体名而放弃、输出 NA。但最终 reader 依然正确答出了"**Chen Zheng**" (EM=1)——因为它同时拿到了这一跳**原始证据全文**(不只是失败的短答案),重新读一遍原始问题 之后,能直接从原文里找到答案,不完全依赖那个失败的中间步骤。

这就是咱们这套 **prompt 格式的核心优势:信号冗余、结构化脚手架**——每一跳同时给 reader 三样 东西(子问题、原始证据全文、这一跳的短答案),哪怕某一样出问题(短答案抽取失败、子问题措辞有 误),reader 还有别的信号可以兜底;而且"推理轨迹"这个结构化格式,相当于替 reader 把"每一跳该 关注什么、上一跳查到了什么"都预先梳理好了,reader 只需要做"综合已经梳理好的信息、回答原始问题" 这一件相对简单的事。

GRITHopper 的 reader 完全没有这套脚手架——它是无需拆解的方法,压根不产生子问题、也不产生 逐跳短答案,能给 reader 的只有一份"按检索顺序排列的原始文档列表":

```
Original question: Who led the military expedition in the province that borders Zhejiang to the south?
```

```
Retrieved documents (in retrieval order):
```

```
Document 1: Zhejiang. Zhejiang ... is bordered by ... Fujian province to the south ...
```

```
Document 2: Hokkien. In 677 ..., Chen Zheng ... led a military expedition to pacify the rebellion in Fujian. ... In 885, ... Wang Chao and Wang Shenzhi, led a military expedition force to pacify the Huang Chao rebellion. ...
```

```
Answer the original question.
```

即使每一步都精确检索到了 gold(这道题就是),reader 依然要**自己从头捋一遍"哪个段落回答哪一跳、这些段落之间怎么串联"**这整套推理,咱们的 pipeline 已经替它做完了这件事(拆子问题、每跳抽取 短答案),GRITHopper 完全没做,全部推给了最后这一次生成。碰到像上面《Hokkien》这种一段话里其实 写了两件相似事情(两次不同年代的"military expedition")的段落,没有中间步骤引导的 reader 更容易 选错——这道题恰好 GRITHopper 答对了,但整体统计上(2.4 节)它的答案准确率明显偏低,说明这类情况 不是个例。

是不是**"因为没有拆解"**——是,而且这是 GRITHopper 论文自己的核心设计选择("Decomposition- Free"就是它的卖点):不依赖拆解模型,换来的好处是不用担心拆解质量拖累检索、泛化到分布外数据更 稳(这也是它检索分数高的原因之一,见上表前的投入成本对比);但代价是把"拆解+分步作答"这部分 工作能力从检索阶段完全拿掉,全部推给最后一次生成去补——检索端拿到的收益,在生成端被更大程度地 吃掉了。ChainRAG 虽然会拆子问题、也生成逐跳答案,但因为图扩展塞进 context 的信息太杂(2.5.1 节),同样没能把"拆解"这个环节的潜力真正发挥出来。咱们的方法是三个里**唯一一把"精确检索 + 结构化 拆解 + 多信号冗余"**这三件事同时做到位的,这大概率是最终 EM/F1 全面领先的根本原因,不只是检索 分数领先这一个维度。

总结一句话:三个方法分别代表三种取舍——ChainRAG(零训练、图扩展兜底、答案合成粗糙)、GRITHopper(重训练换检索精度、无拆解导致答案合成负担全部后移)、D 组(轻量训练的 gate 校正

- 完整的拆解与多信号冗余 reader),后者在检索投入远小于 GRITHopper 的情况下,靠"检索够用 + 推理结构完整"拿到了三个数据集上最好的最终答案效果。